

Sequential Decision-making under Uncertainty

Nan Ye

School of Mathematics and Physics
University of Queensland

sequential decision making under uncertainty

“Mama always said that life was like a box of chocolates.
You never know what you’re gonna get.”

Forrest Gump



optimal decisions are often elusive (analytically and computationally)
good decisions are often computable (via iterative methods)

Sequential Decision Making



ad placement



play Go (DeepMind)



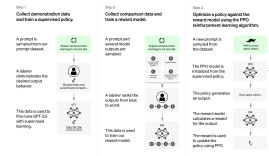
play StarCraft (DeepMind)



robot control (Boston Dynamics)



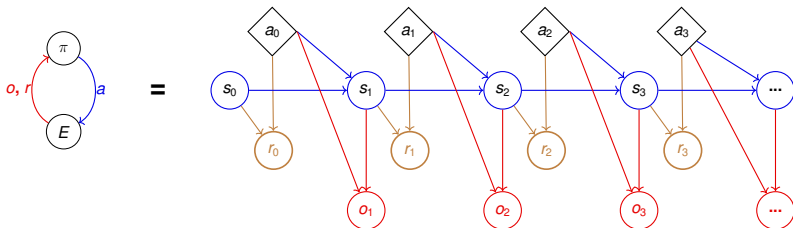
autonomous car (Waymo)



ChatGPT (OpenAI)

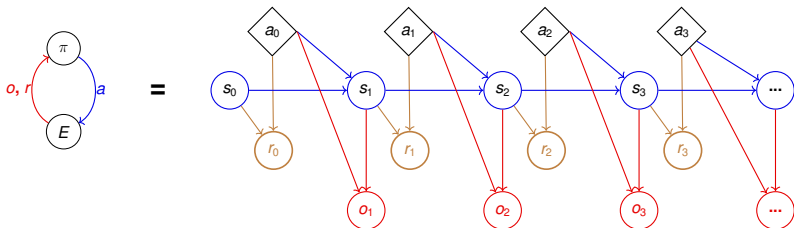
many others: algorithm discovery, ride-hailing order dispatching, dialogue systems, healthcare, energy, ...

General framework



an agent π interacting with an environment E

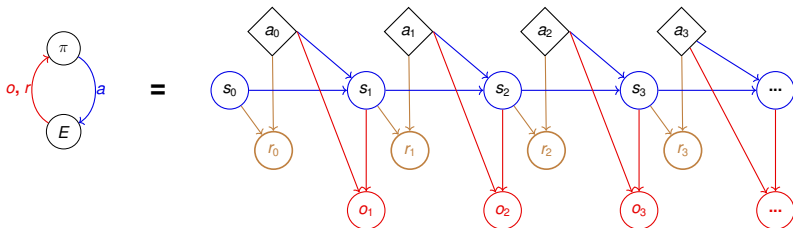
General framework



an agent π interacting with an environment E

at each time step, agent executes an action a , receives an observation o and a reward r , and environment state evolves

General framework

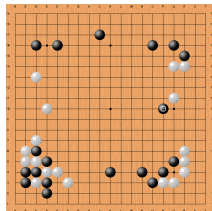


an agent π interacting with an environment E

at each time step, agent executes an action a , receives an observation o and a reward r , and environment state evolves

Example. Playing Go against an opponent

- state s_t : board position
- action a_t : a valid move
- observation o_t : s_t
- reward r_t : 0, except +1 for win and -1 for loss
- transition from s_t to s_{t+1} : determined by the opponent



Example. Tiger



- A tiger is either behind the left door or the right door, with equal probability.
- At time t , either:
 - Listen:
 - ▶ get a reward $r_t = -1$, and
 - ▶ hear a sound from the door with a tiger behind with probability 0.85
 - Open a door
 - ▶ get a reward $r_t = 10$ if tiger is not behind it, and $r_t = -100$ otherwise
 - ▶ no observation, and the game resets.

Example. Tiger



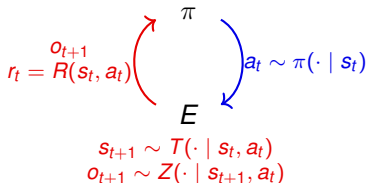
- A tiger is either behind the left door or the right door, with equal probability.
- At time t , either:
 - Listen:
 - ▶ get a reward $r_t = -1$, and
 - ▶ hear a sound from the door with a tiger behind with probability 0.85
 - Open a door
 - ▶ get a reward $r_t = 10$ if tiger is not behind it, and $r_t = -100$ otherwise
 - ▶ no observation, and the game resets.
- Optimal strategy π for maximizing $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi]$ with $\gamma = 0.95$
listen until a door is heard two times more than the other, then open it

POMDPs

Partially Observable Markov Decision Processes

POMDP $\mathcal{M} = (b_0, S, A, O, T, Z, R, \gamma)$

- initial state distribution b_0
- state/action/observation space $S/A/O$
- transition model $T(s' \mid s, a)$
- observation function $Z(o \mid s', a)$
- reward function $R(s, a)$
- discount factor $\gamma \in [0, 1]$



Planning: given $\mathcal{M}$, find an optimal policy $\pi : \mathcal{B} \rightarrow A$ such that

$$V_\pi(b) \geq V_{\pi'}(b) \text{ for any belief } b \text{ and any policy } \pi' : \mathcal{B} \rightarrow A$$

where $V_\pi(b) = \mathbb{E}(\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 \sim b, \pi)$ is the value of π starting from initial belief b , and $\mathcal{B}$ is the set of beliefs (probability distributions on S).

Iterative Refinement

Exact analytical solutions are rarely available.

Many solution methods are computational and they iteratively refine a solution.

An iterative method can be written in the form $x_{t+1} = H_t(x_t)$

- x is the solution
 - this may be a policy or its value or both
 - common representations: tables, parametric functions (e.g., neural networks), trees
- refinement operator H_t uses model/interactions
 - this may be deterministic or stochastic
 - this may be defined explicitly or implicitly

Algorithms are scaling up

- 1990's: tens of states is challenging.
- 2000's: offline point-based methods
PBVI (thousands of states), SARSOP (hundreds of thousands of states)
- after 2010: online belief tree search
POMCP, GPS-ABT, DESPOT, POMCPOW, ADVT, LCEOPT, ...

Offline algorithms plan for **all** beliefs

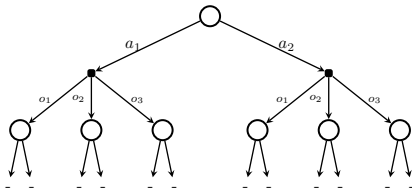
- Plan first, execute later.
- Planning is often slow, but execution is often fast.
- Generally do not exploit the local structure in the search space.

Online algorithms plan for **current** belief

- Interleave planning and execution.
- Possibility of benefiting from local structure in the search space.

Belief-tree Search

- A belief tree encapsulates all possible strategies



- root node is the initial belief
- following an action a from a belief node b and observing o leads to a belief node representing the updated belief
- Naive belief tree search
 - truncate the tree to a finite depth D (#beliefs = $O(|A|^D |O|^D)$)
 - estimate values of leaf beliefs
 - compute action values in a bottom-up manner
- Many online algorithms construct an iteratively refined approximation to the belief tree

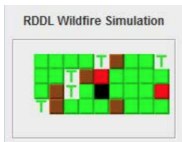
DESPOT

Joint work with Somani, Hsu and Lee

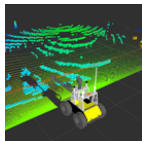
- An anytime online Monte Carlo algorithm for POMDPs with discrete actions and observations.
- It searches a sparse approximation to the belief tree.
- Provably near-optimal.
- Good empirical performance



self-driving



IPPC'14 winner



HRATC'15 winner

Key idea

- Evaluate policies on K *shared scenarios* and penalize large policies to prevent overfitting.
- Specifically, we can rewrite the value of a policy as an expectation over scenarios

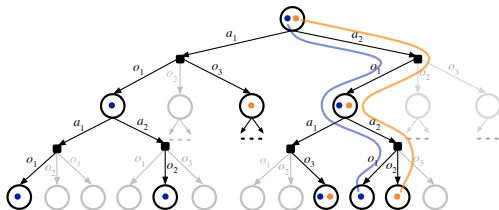
$$V_{\pi}(b) = \mathbb{E}(V_{\pi,\phi}),$$

where

- $\phi = (s_0, \phi_0, \phi_1, \dots)$ is a random scenario with $s_0 \sim b_0$ and each ϕ_t independently sampled from $U(0, 1)$.
- $V_{\pi,\phi}$ is the total discounted reward $\sum_{t=0}^{\infty} \gamma^t r_t$ obtained when simulating π on scenario ϕ (i.e., starting with s_0 , and use random numbers from ϕ to sample next states and observations)
- We can first sample K scenarios $\phi_1, \dots, \phi_K$, then for any policy π , approximate $V_{\pi}(b_0)$ with

$$\hat{V}_{\pi}(b_0) = \frac{1}{K} \sum_{i=1}^K V_{\pi,\phi_i}.$$

- Considering K scenarios and a depth of D , we obtain a sparse approximate belief tree with $O(|A|^D K)$ belief nodes



- Use dynamic programming to optimize $\hat{V}_\pi(b_0) - \lambda|\pi|$.

Optimal regularized policy is near-optimal

Let π^* be an optimal policy at a belief b_0 . For any $\tau, \alpha \in (0, 1)$, for any DESPOT that has height D and is constructed from K randomly sampled scenarios for belief b_0 , if π is a policy derived from the DESPOT maximizing

$$\hat{V}_\pi(b_0) - \lambda |\pi|$$

with $\lambda = \frac{R_{\max}}{(1-\alpha)(1-\gamma)} \cdot \frac{\ln(KD|A||Z|)}{\alpha K}$, then with probability at least $1 - \tau$,

$$V_\pi(b_0) \geq \frac{1-\alpha}{1+\alpha} V_{\pi^*}(b_0) - \frac{R_{\max}}{(1+\alpha)(1-\gamma)} \left(\frac{\ln(8/\tau) + |\pi^*| \ln(KD|A||Z|)}{\alpha K} + (1-\alpha) \left(\sqrt{\frac{2 \ln(2/\tau)}{K}} + \gamma^D \right) \right),$$

$O(|\pi^*| \ln |\pi^*|)$ scenarios required for near-optimal performance.

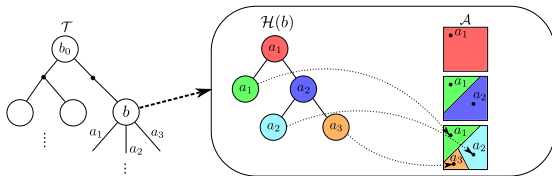
ADVT

Joint work with Hoerger, Kurniawati and Kroese

- An anytime online Monte Carlo algorithm for POMDPs with continuous actions.
- Example robotic tasks



Idea



- maintains an adaptively refined hierarchical partition, called Voronoi tree, of the action space for each node
- starts with a single belief node tree, and performs simulations to grow it to better approximations for the belief tree
- for each simulation, each step chooses an action from the representative actions in the Voronoi tree using a modified UCB1 strategy
- a cell in a Voronoi tree is randomly split whenever it has been sufficiently explored
- continuous observations are handled using a progressive widening strategy that samples a new observation when current observations have been sufficiently explored.

LCEOPT

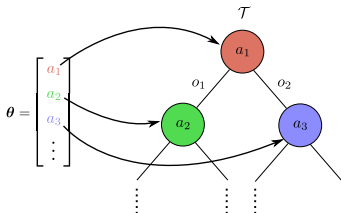
Joint work with Hoerger, Kurniawati and Kroese

- An anytime online algorithm for POMDPs with continuous actions and discrete observations.
- Key idea: use a *lazy* cross-entropy (CE) method to solve

$$\max_{\pi} V_{\pi}(b)$$

over the space of policies represented as policy trees.

- Policy tree allows a finite-dimensional vector representation for a (finite-horizon) policy



CE method for optimization

$\max_x f(x)$, where $f : \mathbb{R}^d \rightarrow \mathbb{R}$

- Start with an initial distribution D_0 on solutions
- At iteration t
 - Sample candidate solutions from D_t and evaluate them
 - Keep the top solutions, and fit a distribution D_{t+1} for them

Lazy CE method

- Naively, CE method can be now be directly applied to solve $\max_{\pi} V_{\pi}(b)$, but this is too slow.
- Evaluating a policy π by simulation often does not require knowing all of π 's parameters $\Rightarrow$
 - we only sample the encountered policy parameters during simulation
 - we update the policy distribution based on policy vectors with missing values

Reinforcement Learning

- Learning an optimal policy from offline interaction data
- Learning interpretable decision tree policies (Joel Woodfield)

This Talk

- POMDPs
- Iterative refinement
- Several approximate solvers
 - DESPOT: sparse belief tree + regularization
 - ADVT: action space discretization using Voronoi tree
 - LCEOPT: lazy cross-entropy method